Name of the Student	**G.Venkatasai**
Reg. No	**192425388**
GitHub Link	https://github.com/venkatasai-eng/MLA0403-Deep-learning

SIMATS ENGINEERING

CO4 - ASSESSMENT TOOL 1

Design and Develop Model

COURSE NAME: Deep Learning COURSE CODE: MLA04 SLOT : A

COURSE OUTCOME COVERED

CO 4: Students will be able to understand and apply probabilistic graphical models including Bayesian Networks, Markov Random Fields, Hidden Markov Models, and Conditional Random Fields. They will learn how to represent complex probabilistic relationships among variables and perform inference and learning in graphical models. Students will be able to use these models for reasoning under uncertainty and solving real-world decision-making problems. BL-6

Course Outcome Weightage: 10% Duration: 90 minutes Assessment Tool Weightage: 30% Mark : 25

Code of Conduct:
I Guggilla Venkatasai(192425388) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:  **Name of the student: G.Venkatasai**

Criteria	Problem Understanding & Requirement Analysis **(2)**	Model Architecture & Design **(2.5)**	Application of Deep Learning Techniques **(2.5)**	Model Development & Implementation Strategy **(2)**	Performance Analysis & Justification ***(1)***	**Total (10)**
Weightage	***20 %***	***30%***	***25%***	***15 %***	***10 %***	**100%**

Q1						
Q2						
Q3						
Q4						
Q5						
Total						

QUESTION 1: Transfer Learning Model for Medical X-Ray Classification (2,000 Images)

1. Detailed Problem Analysis & Constraints

In medical imaging diagnostic tasks, training deep convolutional networks from scratch presents severe challenges due to high data scarcity. Here, the hospital possesses only 2,000 labeled X-ray images (Normal vs. Abnormal). If a deep neural network such as ResNet-50 (containing over 23 million parameters) were trained from scratch on 2,000 images, the model would rapidly memorize training samples, causing severe overfitting and poor generalization to unseen patient scans.

Transfer learning mitigates this bottleneck by utilizing weights pre-trained on large-scale benchmark datasets like ImageNet (1.4M images). ImageNet features enable the model to transfer low-level visual representations directly to the medical domain.

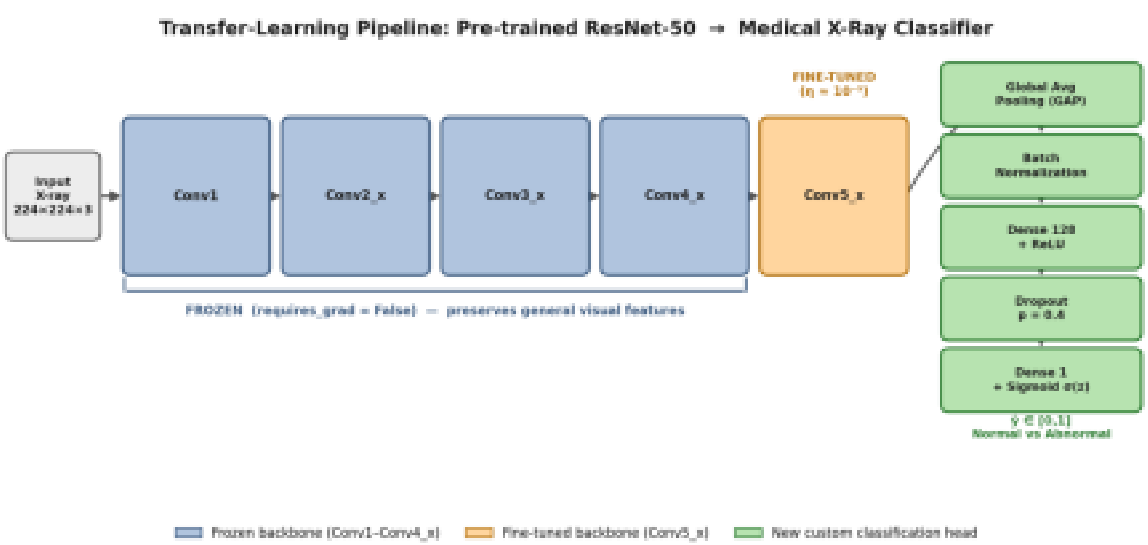

Fig. Q1 – ResNet-50 transfer-learning pipeline: frozen backbone, fine-tuned Conv5_x, and custom classification head

2. ResNet Architecture Breakdown & Layer Freezing Strategy

ResNet models rely on residual learning blocks with shortcut skip connections ($y = F(x, \{W_i\}) + x$), enabling efficient gradient flow through deep network depth. To optimize performance on 2,000 X-ray images, we divide the pre-trained ResNet-50 into three functional architectural components:

Layer Modification Blueprint: • Frozen Lower Layers (Conv1 to Conv4_x): Frozen to preserve general visual features (edges, textures, spatial gradients). • Fine-Tuned Upper Layers (Conv5_x): Unfrozen to learn domain-specific medical features (radiological shadow density, lesions). • Replaced Classification Head: Original 1,000-class ImageNet fully connected layer replaced with a custom binary classification pipeline.

Layer Group	Action	Rationale & Mathematical Mechanics
Conv1 - Conv4_x	Freeze (Set requires_grad = False)	Prevents destruction of universal visual features. Reduces trainable parameters by ~80%, accelerating training and preventing overfitting.
Conv5_x	Unfreeze (Set requires_grad = True)	Adapts high-level abstract representations to radiological anomalies with a small fine tuning learning rate ($\eta \approx 10^{-5}$).
Output Head	Replace with Custom Layers	Maps abstract spatial activations directly to binary Normal vs. Abnormal target probabilities.

3. Custom Classification Head Design

The original fully connected layer of ResNet is stripped away and replaced with the following layers:

- **1. Global Average Pooling (GAP) Layer:** Replaces flattening to convert 7x7x2048 feature maps into a single 2048-dimensional vector, drastically reducing parameter count and preventing spatial overfitting.
- **2. Batch Normalization Layer:** Normalizes activations, stabilizing internal covariate shift. • **3. Fully Connected Dense Layer:** Consists of 128 units with ReLU activation ($f(z) = \max(0, z)$). • **4. Dropout Layer:** Set at rate $p = 0.4$, randomly deactivating 40% of neurons during training passes to prevent feature co-adaptation.
- **5. Output Layer:** 1 dense neuron with Sigmoid activation ($\sigma(z) = 1 / (1 + e^{-z})$) producing predicted probability $\hat{y} \in [0, 1]$.

4. Comprehensive Model Training Strategy

A. Medical Data Augmentation Pipeline

Standard data augmentations that distort diagnostic properties (such as extreme shearing or color jitter) are avoided. Safe radiologic augmentations include:

- **Random Small Rotations:** ±10° to account for minor patient positioning shifts. • **Horizontal Flipping:** Applied only if non-anatomical symmetry holds.
- **Brightness & Contrast Adjustments:** ±10% shift to simulate different X-ray machine exposure levels.

B. Objective Function & Optimization Mechanics

The model is trained using Binary Cross-Entropy Loss with L2 Weight Decay:

Loss Formula:

$$L\_BCE = - (1/N) * \sum [y\_i * \log(\hat{y}\_i) + (1 - y\_i) * \log(1 - \hat{y}\_i)] + \lambda \|W\|_2^2$$

We utilize the Adam Optimizer with differential learning rates (Discriminative Layer Training):

- **Custom FC Head Learning Rate:** $\eta\_head = 10^{-3}$ for rapid convergence of randomly initialized weights.
- **Fine-Tuned Conv5_x Learning Rate:** $\eta\_backbone = 10^{-5}$ to gently adjust residual weights without destroying pre-trained features.

5. Evaluation & Performance Analysis

- **Stratified 5-Fold Cross-Validation:** Ensures equal proportions of normal and abnormal X-rays across train and validation splits.
- **Primary Metrics:** Area Under the Receiver Operating Characteristic Curve (ROC-AUC), Sensitivity (Recall for abnormal detection), and Specificity.
- **Early Stopping:** Monitors validation loss and halts training if validation loss fails to decrease for 8 consecutive epochs.

QUESTION 2: DenseNet-Based Classification Model for Plant Disease Identification

1. Detailed Problem Analysis & Background

An agricultural organization requires an automated system to classify five types of plant leaf diseases using limited leaf images. Agricultural images contain localized patterns such as minor lesions, vein discoloration, and edge spots. Traditional feed-forward convolutional networks require millions of parameters to learn hierarchical features, leading to vanishing gradient problems and redundant feature re-learning when training on constrained datasets.

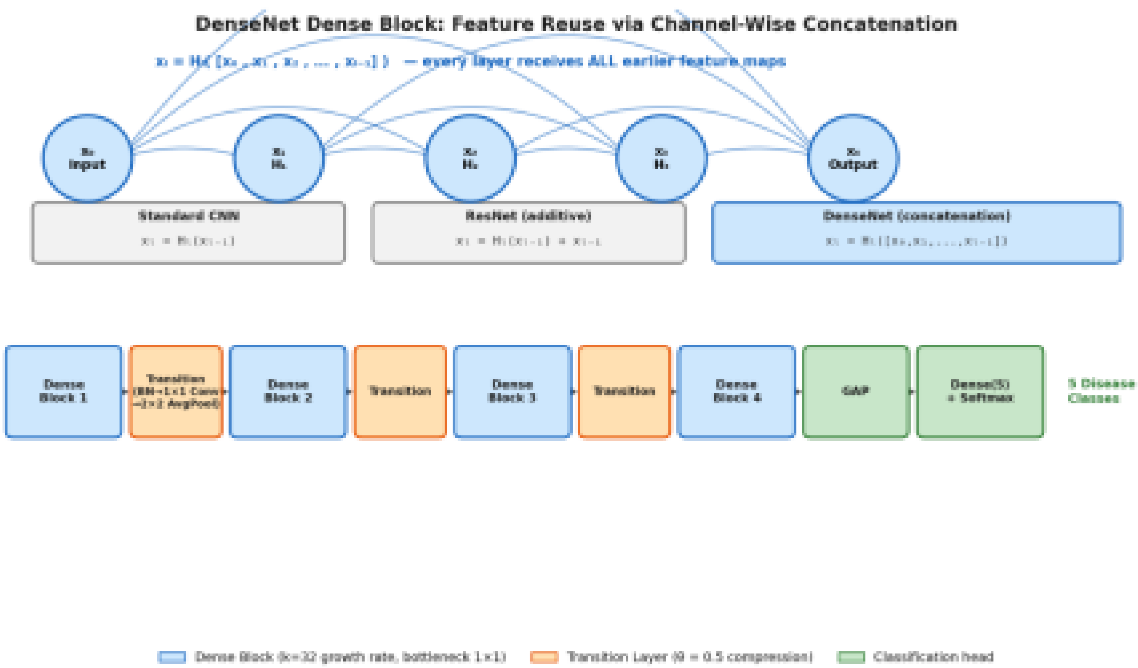

Fig. Q2 – DenseNet dense-block connectivity, transition layers, and classification pipeline

2. DenseNet Architecture & Feature Reuse Mechanics

DenseNet (Densely Connected Convolutional Networks) resolves parameter inefficiency by introducing direct connections from every layer to all subsequent layers. This architecture contrasts directly with Standard CNNs and ResNets:

Connection Formulations Across Architectures:

- Standard CNN: $x_l = H_l(x_{l-1})$ (Information passes strictly sequentially).
- ResNet: $x_l = H_l(x_{l-1}) + x_{l-1}$ (Additive shortcut connection).
- DenseNet: $x_l = H_l([x_0, x_1, x_2, ..., x_{l-1}])$ (Channel-wise feature concatenation). Here, $H_l(\cdot)$ represents a non-linear composite transformation consisting of Batch Normalization (BN), ReLU, and a 3x3 Convolution layer.

3. Why Feature Reuse Benefits Limited Dataset Training

Mechanism	**How It Works in Leaf Disease Classification**	**Impact on Model Performance**
Direct Feature Concatenation	Early layers extract elementary features (leaf edges, color spots). DenseNet preserves these feature maps directly into deeper layers without attenuation.	Prevents deep layers from re learning low-level leaf features, maximizing information efficiency.
Improved Gradient Flow	Loss gradients flow directly back to initial layers via short paths without passing through complex non-linear weight matrices.	Mitigates vanishing gradient issues completely during backpropagation.
Compact Parameter Footprint	Due to feature concatenation, individual layers can be very narrow (adding only k feature maps per layer).	Drastically reduces parameter count, acting as a built-in regularizer against overfitting.

4. Complete Structural Design of the DenseNet Model

The proposed plant disease classifier is structured around four Dense Blocks interconnected via Transition Layers:

A. Growth Rate (k) & Bottleneck Layers

The hyperparameter Growth Rate (k) defines how many feature channels are concatenated at each layer (e.g., $k = 32$). To reduce computational complexity, a 1x1 convolution (Bottleneck layer) is placed before each 3x3 convolution, producing 4k feature maps.

B. Transition Layers (Dimensionality Control)

Between consecutive Dense Blocks, a Transition Layer is inserted to perform downsampling and feature compression:

• **Components:** Batch Normalization → 1x1 Convolution → 2x2 Average Pooling layer. • **Compression Factor (θ):** Reduces output feature maps to $\lfloor \theta \cdot m \rfloor$ (where $\theta = 0.5$). This keeps spatial dimensions and memory usage under control.

C. Output Classification Pipeline for 5 Plant Diseases

• **1. Final Dense Block Output:** Generates concatenated multi-scale feature maps. • **2. Global**

Average Pooling (GAP): Averages spatial activations per feature channel. • **3. Dense Output Layer:** 5 output neurons corresponding to the 5 plant disease categories. • **4. Activation Function:** Softmax Activation to compute class probabilities: ŷ_i = e^(z_i) / ∑ e^(z_j).

5. Training & Performance Justification

- **Loss Function:** Categorical Cross-Entropy Loss (L = - ∑ y_i * log(ŷ_i)).
- **Data Augmentation:** Random flipping, affine rotations, and color jitter to account for field lighting conditions.
- **Optimization:** SGD with Nesterov Momentum (0.9), initial learning rate η = 0.1 with cosine annealing learning rate schedule.

QUESTION 3: LSTM-Based Model for Student Academic Performance Prediction

1. Detailed Problem Analysis & Sequential Nature

A university aims to predict a student's final academic performance category (High, Average, Low) using temporal tracking data collected across multiple weeks. The inputs recorded per week t include: weekly attendance percentage, assignment marks, internal assessment scores, and previous examination scores.

Because student performance evolves dynamically over time, standard feedforward artificial neural networks cannot process sequence dependencies effectively. Long Short-Term Memory (LSTM) networks are designed specifically for multivariate sequential data, overcoming the vanishing gradient problem inherent in standard Recurrent Neural Networks (RNNs).

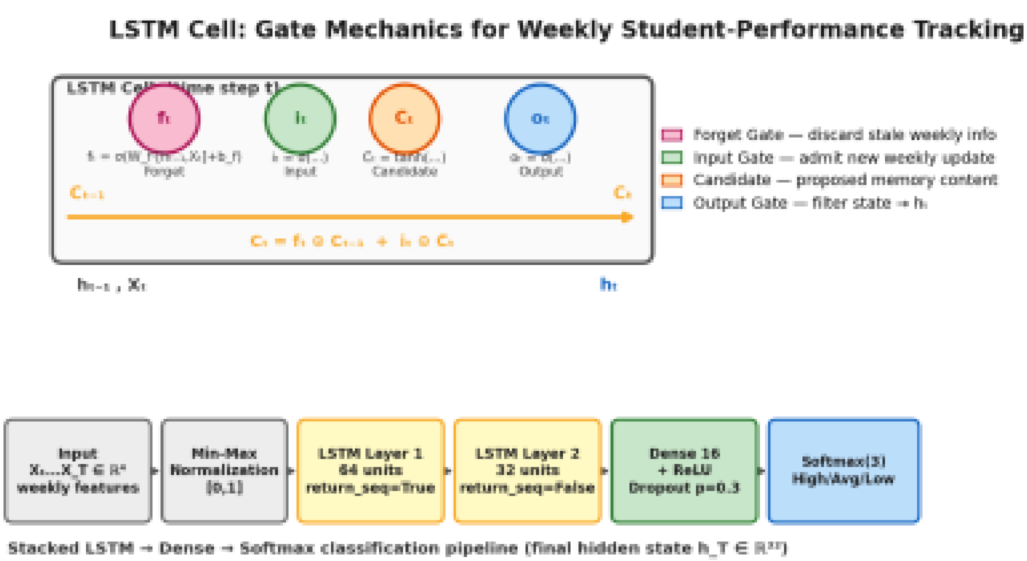

Fig. Q3 – LSTM gate mechanics and the stacked LSTM → Dense → Softmax pipeline

2. Vector Formalization & Input Representation

For a given student monitored over T weeks, the input is represented as a sequence of feature vectors:

Input Sequence Representation:
$X = (X_1, X_2, ..., X\_t, ..., X\_T)$, where $X\_t = [$ Attendance_t, Assignment_Score_t, Internal_Assessment_t, Previous_Exam_Score_t $]^T \in \mathbb{R}^4$

3. Detailed Mathematical Breakdown of LSTM Internal Gates

At each weekly time step t, the LSTM cell receives the current vector X_t, the previous hidden state h_{t-1}, and the previous cell state C_{t-1}. It processes information through three internal gating mechanisms:

LSTM Gate Component	**Mathematical Equation**	**Functional Explanation in Student Tracking**
Forget Gate (f_t)	f_t = σ(W_f · [h_{t-1}, X_t] + b_f)	Controls how much past information to discard (e.g., discounting an old bad week if recent performance improved drastically).
Input Gate (i_t) & Candidate (C̃_t)	i_t = σ(W_i · [h_{t-1}, X_t] + b_i) C̃_t = tanh(W_c · [h_{t-1}, X_t] + b_c)	Determines what new weekly academic updates should be incorporated into the long-term cell memory state.
Cell State Update (C_t)	C_t = f_t ⊙ C_{t-1} + i_t ⊙ C̃_t	Updates the internal constant error memory state by forgetting past noise and adding new weighted information.
Output Gate (o_t) & Hidden (h_t)	o_t = σ(W_o · [h_{t-1}, X_t] + b_o) h_t = o_t ⊙ tanh(C_t)	Filters the updated internal state to output a hidden state representation h_t for current evaluation.

4. End-to-End System Architecture

The complete predictive architecture converts multi-week temporal student data into a single performance category through the following processing pipeline:

- **1. Input Normalization Layer:** Mini-Max scaling applied to bring feature ranges (e.g., attendance percentage vs. test scores) into a uniform range [0, 1].

- **2. Stacked LSTM Layer:** Layer 1 consists of 64 hidden units (return_sequences = True). Layer 2 consists of 32 hidden units (return_sequences = False), returning the final summary state vector $h\_T \in \mathbb{R}^{32}$.
- **3. Dense Classification Layer:** Fully connected dense layer (16 units, ReLU activation) + Dropout ($p = 0.3$).
- **4. Softmax Output Layer:** 3 units predicting performance probabilities for High, Average, and Low performance.

Final Output Equation:
$\hat{y} = \text{Softmax}(W\_out \cdot h\_T + b\_out)$

5. Model Training & Evaluation Metrics

- **Loss Function:** Categorical Cross-Entropy Loss optimized via Adam optimizer ($\eta = 0.001$). • **Handling Sequential Imbalance:** Synthetic Minority Over-sampling (SMOTE-NC) or class weights applied if low-performing students form a minority class.
- **Evaluation Metrics:** Macro F1-Score, Confusion Matrix, and Temporal Precision at Week k (evaluating model accuracy early in the semester).

QUESTION 4: Bidirectional RNN Model for Customer Review Sentiment Analysis

1. Detailed Problem Analysis & Contextual Dependencies

An organization needs to classify customer text reviews into Positive, Neutral, or Negative sentiment. In natural language processing (NLP), word meaning depends heavily on surrounding context. Unidirectional RNNs process sentences left-to-right, meaning when the model reads word t, it only knows previous words (t-1, t-2) and lacks information about upcoming words (t+1, t+2).

For example, in the review phrase 'Not only was the service bad, but the food was also terrible' versus 'Not bad at all, in fact, it was excellent!', the sentiment of 'bad' depends directly on subsequent words. A Bidirectional Recurrent Neural Network (Bi-RNN) addresses this limitation by processing the input text simultaneously in both forward and backward directions.

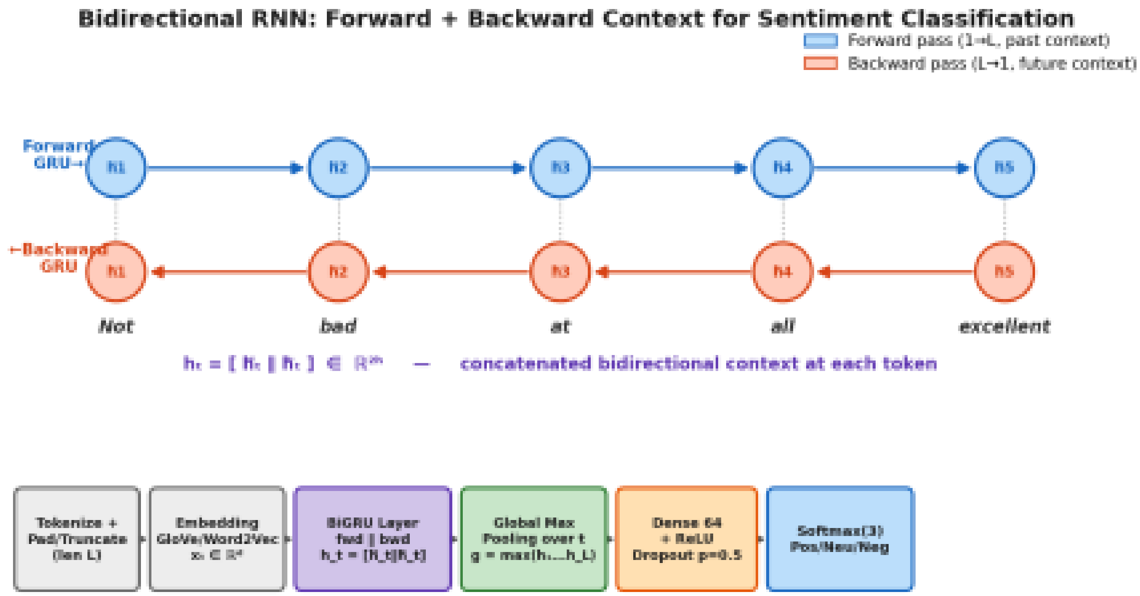

Fig. Q4 – Bidirectional RNN forward/backward context and sentiment classification pipeline

2. Major Processing Stages: Input Text to Prediction

Stage 1: Text Preprocessing & Cleaning

- **Text Normalization:** Lowercasing text, stripping special symbols/punctuation, and handling contractions (e.g., 'don't' → 'do not').
- **Tokenization:** Splitting raw sentences into individual word tokens.
- **Padding & Truncation:** Standardizing review lengths to a maximum sequence length L using zero-padding.

Stage 2: Word Embedding Representation Layer

Words are converted from discrete integer tokens into continuous vector space using pre-trained word embeddings (e.g., GloVe or Word2Vec) or a trainable Embedding Layer:

Embedding Output:
$X = (x_1, x_2, ..., x\_t, ..., x\_L)$, where $x\_t \in \mathbb{R}^d$ ($d = 100$ or 300 dimensional vector)

Stage 3: Bidirectional Recurrent Layer Mechanics

The core layer consists of two independent recurrent layers running in parallel over input vector x_t:

Processing Direction	Mathematical Formulation	Context Captured
Forward Direction ($\vec{h}\_t$)	$\vec{h}\_t = GRU\_fwd(x\_t, \vec{h}\_{t\ 1})$	Processes text sequentially from index 1 → L. Captures historical left-to-right context.

Backward Direction ($\overleftarrow{h}_t$)	$\overleftarrow{h}_t = GRU\_bwd(x_t, \overleftarrow{h}_{t+1})$	Processes text in reverse sequence from index L → 1. Captures future right-to-left context.
Concatenation (h_t)	$h_t = [\overrightarrow{h}_t \parallel \overleftarrow{h}_t] \in \mathbb{R}^{2h}$	Combines forward and backward vectors at time step t to form a complete bidirectional context representation.

Stage 4: Feature Pooling & Dense Classification Head

- **1. Global Max Pooling Over Time:** Extracts the most salient features across all token hidden vectors h_t: $g = \max(h_1, h_2, ..., h_L)$.
- **2. Dense Fully Connected Layer:** 64 units with ReLU activation, followed by Dropout ($p = 0.5$) to prevent overfitting.
- **3. Softmax Classification Output Layer:** 3 output units mapping probabilities across Positive, Neutral, and Negative classes: $P(Y = c \mid X) = e^{z_c} / \sum e^{z_k}$.

3. Training Parameters & Loss Function

• **Loss Function:** Categorical Cross-Entropy Loss: $L = - \sum y_c * \log(\hat{y}_c)$. • **Optimization:** Adam Optimizer ($\eta = 10^{-3}$) with Gradient Clipping (max norm = 1.0) to prevent exploding gradients.

QUESTION 5: Encoder–Decoder Model for English-to-Hindi Machine Translation

1. Detailed Problem Analysis & Sequence-to-Sequence Concept

Translating sentences from English (source) to Hindi (target) presents unique sequence-to-sequence (Seq2Seq) challenges:

- **Variable Input and Output Lengths:** An English sentence with 8 words may translate into a Hindi sentence with 10 words.
- **Structural Word Order Reordering:** English follows Subject-Verb-Object (SVO) ordering (e.g., 'He reads a book'), whereas Hindi follows Subject-Object-Verb (SOV) ordering (e.g., 'वह एक ककताब पढ़ता है').

To address this, an Encoder–Decoder architecture is used. The model reads the entire source sentence first, encodes its semantic meaning into a hidden representation, and then generates the target translated sentence step-by-step auto-regressively.

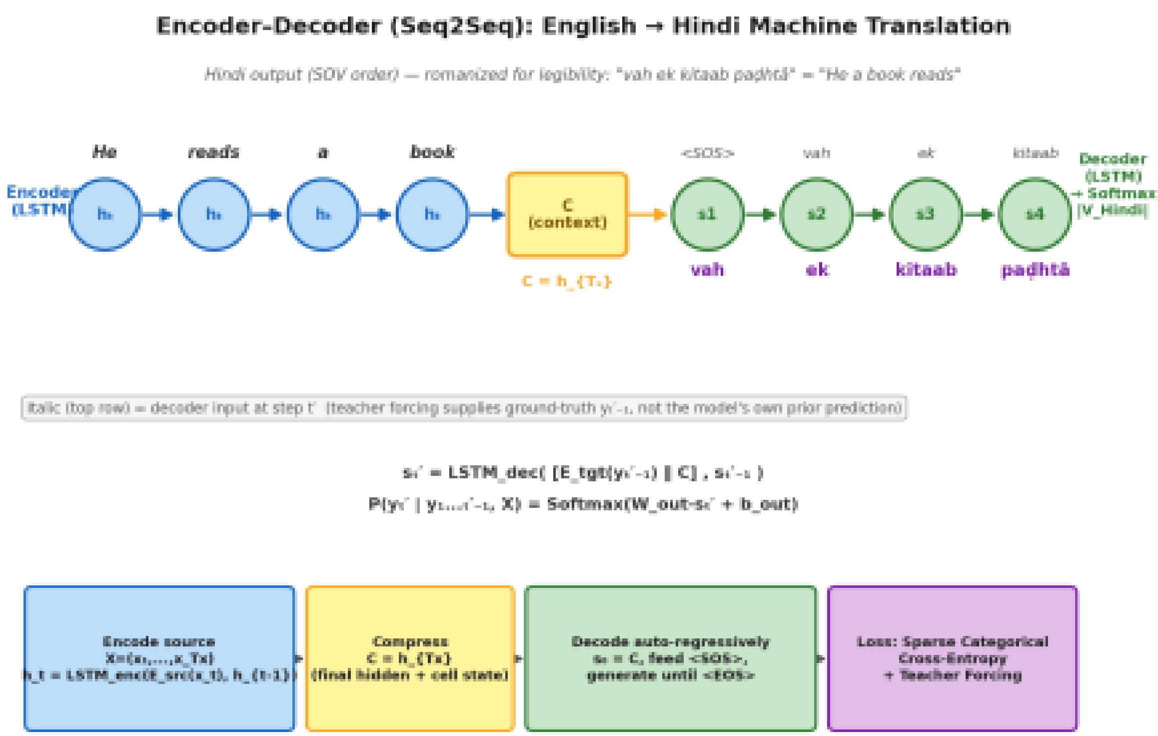

Fig. Q5 – Encoder–Decoder sequence-to-sequence architecture for English-to-Hindi translation

2. Key Architectural Components & Their Roles

A. Encoder Architecture & Role

The Encoder consists of a multi-layer Recurrent Neural Network (LSTM or GRU). Its function is to process the English input sequence $X = (x_1, x_2, ..., x\_Tx)$ step-by-step and compress its full context into a fixed-dimensional continuous vector space.

Encoder Recurrent Update:
h_t = LSTM_enc(E_src(x_t), h_{t-1})
Where E_src(x_t) is the embedding vector for English token x_t, and h_t is the encoder hidden state at step t.

B. Hidden Representation (Context Vector C)

The Context Vector (C) acts as the central information bridge between the Encoder and the Decoder. It is derived from the final hidden state (and cell state) of the encoder after reading the last input token at step T_x:

Context Vector Formula:
C = h_{T_x}

This single vector summarizes the global semantic meaning and grammatical nuance of the entire input English sentence.

C. Decoder Architecture & Role

The Decoder is a secondary Recurrent Neural Network (LSTM or GRU) that generates the translated Hindi sequence $Y = (y_1, y_2, ..., y\_Ty)$ auto-regressively token by token.

- **Initialization:** The initial hidden state of the decoder is initialized directly using the Context Vector: $s_0 = C$.
- **Start Token Handling:** Translation begins by feeding a special Start-of-Sequence token (<SOS>) as the initial decoder input y_0.
- **Sequential Decoding Loop:** At step t', the decoder calculates its updated hidden state s_{t'} using the context vector, the previous decoder state s_{t'-1}, and the previously generated Hindi token y_{t'-1}: s_{t'} = LSTM_dec([E_tgt(y_{t'-1}) ‖ C], s_{t'-1}).

D. Output Layer & Probability Distribution

The hidden state s_{t'} passes into a dense output linear layer followed by a Softmax layer scaled over the complete Hindi vocabulary |V_Hindi|. This generates a probability distribution over all possible Hindi words:

Output Probability Distribution:
P(y_{t'} | y_{1...t'-1}, X) = Softmax(W_out · s_{t'} + b_out)

The token with the highest probability is emitted as the predicted Hindi word. Generation continues sequentially until the model outputs an End-of-Sequence token (<EOS>) or reaches maximum target length.

3. Model Training Mechanics & Teacher Forcing

During training, the system uses Teacher Forcing: instead of feeding the model's own predicted token y_{t'-1} into the next decoder step, the ground-truth Hindi target token is supplied directly. This stabilizes training convergence speed.

- **Loss Function:** Sparse Categorical Cross-Entropy calculated per target word step, averaged across sequence lengths.
- **Optimization:** Adam Optimizer with learning rate decay and gradient norm clipping.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding & Requirement Analysis	20%	Comprehensive understanding of real-world problem and constraints	Good understanding with minor gaps in requirements	Basic understanding; some requirements unclear	Poor understanding; misinterprets problem context

Model Architecture & Design	30%	Applies advanced and relevant concepts effectively to solve the problem	Applies appropriate concepts with minor errors	Limited application of concepts	Incorrect or no application of concepts
Application of Deep Learning Techniques	25%	Innovative, practical, and feasible solution aligned with industry standards	Logical and feasible solution with minor gaps	Basic solution with limited feasibility	Unclear or impractical solution
Model Development & Implementation Strategy	15%	Effective use of modern tools, technologies, and real data	Appropriate use of tools with correct implementation	Limited or inconsistent use of tools	Minimal or incorrect use of tools
Performance Analysis & Justification	10%	Thorough analysis with validated results and performance evaluation	Good analysis with reasonable validation	Basic analysis with limited validation	Poor or no analysis